

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Вычислительная математика»
Тема: Решение СЛАУ методом простых итераций

Студент гр. 4343

Толмачев И.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Целью данной работы является практическое освоение метода простых итераций для решения систем линейных алгебраических уравнений. В рамках работы необходимо разработать программу на языке C++, реализующую указанный метод, и провести ее тестирование на матрицах с диагональным преобладанием. Для оценки корректности и эффективности собственной реализации предполагается сравнить полученные результаты с решением, вычисленным с помощью библиотечной функции LAPACK, по таким критериям, как время выполнения, относительная невязка и евклидово расстояние между векторами решений.

Выполнение работы.

В ходе выполнения работы была разработана программа на языке C++, реализующая метод простых итераций — метод Якоби, для решения систем линейных алгебраических уравнений.

Основная вычислительная часть программы сосредоточена в функции *jacobiMethod*. Она принимает матрицу A , вектор b , требуемую точность ε и максимальное число итераций, а возвращает вектор решения, количество выполненных итераций и признак успешной сходимости. Внутри функции сначала выполняется проверка диагональных элементов матрицы на равенство нулю: если модуль какого-либо a_{ii} оказывается меньше 10^{-12} , функция немедленно завершается с отрицательным результатом, поскольку в этом случае итерационная формула не определена. Начальное приближение $x^{(0)}$ инициализируется нулевым вектором.

Итерационный цикл организован с помощью переменной *iterCount*, изменяющейся от единицы до *maxIter*. На каждом шаге цикла выполняется один проход метода Якоби: для каждой строки i вычисляется сумма $\sum_{j \neq i} a_{ij} x_j^{(old)}$, после чего новое значение $x_i^{(new)}$ определяется по формуле:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^k}{a_{ii}}.$$

Вектор нового приближения целиком формируется на

основе компонент предыдущего приближения, что соответствует классической схеме Якоби. После завершения прохода вычисляется бесконечная норма разности $\|x^{(new)} - x^{(old)}\|_{\infty}$ путём нахождения максимального по модулю отклонения среди всех компонент.

Полученное значение нормы сравнивается с заданной точностью ε . Если оно оказывается меньше ε , цикл прерывается, флаг сходимости устанавливается в истинное значение, и функция возвращает текущее приближение $x^{(new)}$ в качестве решения. В противном случае вектор $x^{(old)}$ замещается вектором $x^{(new)}$, и процесс повторяется. Если по достижении *maxIter* сходимость не была достигнута, функция возвращает последнее вычисленное приближение с отрицательным флагом сходимости.

Для оценки качества полученного решения вычисляется вектор невязки $r = Ax - b$ с помощью функции *residual*, выполняющей умножение исходной матрицы на найденный вектор и вычитание правой части. Норма невязки рассчитывается как евклидова длина вектора r посредством функции *norm*. Относительная невязка определяется как отношение нормы невязки к норме вектора правой части. Все результаты, включая размерность системы, время выполнения метода, количество итераций и относительную невязку, выводятся в консоль.

Таким образом, разработанный программный комплекс обеспечивает полный цикл численного эксперимента: от генерации тестовых задач с контролируемыми свойствами до решения методом Якоби и оценки точности полученных результатов.

Исследование.

Для оценки эффективности и точности реализованного метода Якоби было проведено его сравнение с прямыми методами решения СЛАУ на тестовой матрице размерности 1000, обладающей строгим диагональным преобладанием. Сравнение выполнялось с библиотечной функцией LAPACK,

реализующей LU-разложение с выбором ведущего элемента, а также с собственной реализацией метода Гаусса. Результаты экспериментов представлены на рисунках 1 и 2.

```
Размерность системы: 1000
Время метода Якоби: 5679 мкс
Время LAPACK (dgesv): 97826 мкс
Метод Якоби сошёлся за 9 итераций
Относительная невязка Якоби: 1.123748e-13
Относительная невязка LAPACK: 1.819451e-15
Погрешность ||x_jacobi - x_lapack||_2 : 2.065477e-09
```

Рисунок 1 — Сравнение с LAPACK, матрица 1000×1000

На рисунке 1 приведены результаты решения системы методом Якоби и функцией LAPACK. Метод Якоби сошёлся за 9 итераций, что подтверждает высокую скорость сходимости на матрицах с диагональным преобладанием. Время его работы составило 5679 мкс, что примерно в 17 раз меньше, чем время, затраченное LAPACK 97826 мкс. Относительная невязка метода Якоби имеет порядок 10^{-13} , а LAPACK — 10^{-15} . Евклидово расстояние между решениями составляет $2.07 \cdot 10^{-9}$, что говорит о высокой степени совпадения результатов. Таким образом, на хорошо обусловленной задаче метод Якоби обеспечивает приемлемую точность при значительно меньших временных затратах.

```
Размерность системы: 1000
Время метода Гаусса: 101189 мкс
Время LAPACK (dgesv): 93164 мкс
Относительная невязка Гаусса: 1.820050e-15
Относительная невязка LAPACK: 1.819451e-15
```

Рисунок 2 — Сравнение с методом Гаусса, матрица 1000×1000

На рисунке 2 представлены аналогичные результаты для сравнения с собственной реализацией метода Гаусса. Время решения методом Гаусса составило 101189 мкс, что почти в 18 раз превышает время метода Якоби. Относительная невязка Гаусса также находится на уровне 10^{-15} . Оба прямых

метода демонстрируют близкую точность, но итерационный метод Якоби оказывается существенно быстрее на данной задаче. Полученные результаты свидетельствуют о корректной работе разработанной программы и наглядно демонстрируют преимущество итерационного подхода при решении больших систем с диагональным преобладанием.